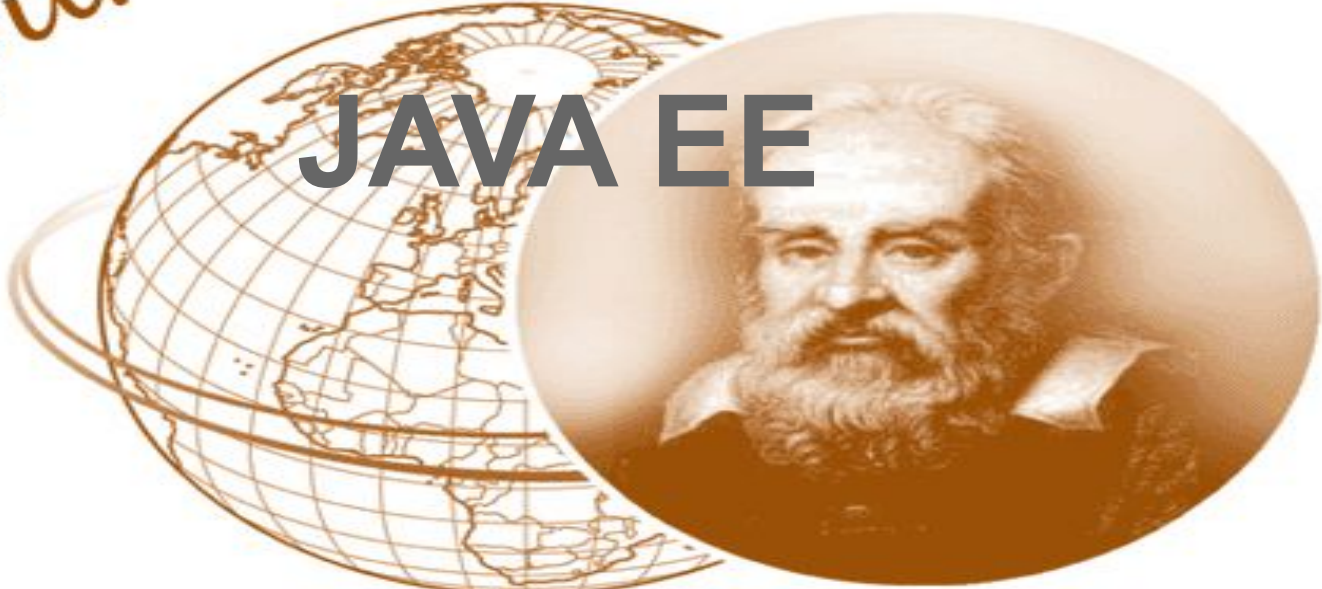


Eppur si muove !

JAVA EE



Samir MILOUDI
samir.miloudi@gmail.com

Institut Galilée

Sommaire

1. Les annotations en JAVA
2. Création d'une application web
 - a. Modèle de données
 - b. Configuration Eclipse
 - c. JSP
 - d. Bean Java
 - e. DAO
 - f. Servlet

Annotations

- Les annotations sont des méta-données permettant d'enrichir la description d'un élément Java.
- Supporté depuis Java 6 (2006).
- Les annotations ont pour but de simplifier le développement.
- Une annotation que vous connaissez déjà : javaDoc
- Le caractère utilisé pour décrire une annotation est le “@”

Annotations - WebServlet

L'API servlet version 3 permet d'utiliser des annotations pour simplifier la gestion d'URL faite via le fichier de configuration web.xml

Pour déclarer l'URL : <http://localhost:8080/demo13/hello>

```
<servlet>
  <servlet-name>SimpleServlet</servlet-name>
  <servlet-class>com.demo.paris13.SimpleServlet</servlet-class>
</servlet>

<servlet-mapping>
  <servlet-name>SimpleServlet</servlet-name>
  <url-pattern>/hello</url-pattern>
</servlet-mapping>
```

```
12 @WebServlet("/hello")
13 public class SimpleServlet extends HttpServlet {
14
```

Annotations - WebInitParam

L'annotation `@WebInitParam` permet de déclarer un paramètre d'initialisation directement dans la servlet.

```
<servlet>
  <servlet-name>SimpleServlet</servlet-name>
  <servlet-class>com.demo.paris13.SimpleServlet</servlet-class>

  <init-param>
    <param-name>name</param-name>
    <param-value>Paris 13</param-value>
  </init-param>
</servlet>
```

web.xml (sans annotation)

```
@WebServlet(urlPatterns= "/hello", initParams = @WebInitParam(name = "name", value="Paris 13"))
public class SimpleServlet extends HttpServlet {
```

SimpleServlet.java avec annotation

Création d'une application web

Pour mieux illustrer les notions rencontrés jusqu'ici, nous allons développer une application simple en Java EE : une todoList.

Spécification fonctionnelle :

- L'utilisateur pourra

 - afficher la liste de toutes les taches

 - créer une tache

 - mettre à jour une tache

 - supprimer une tache

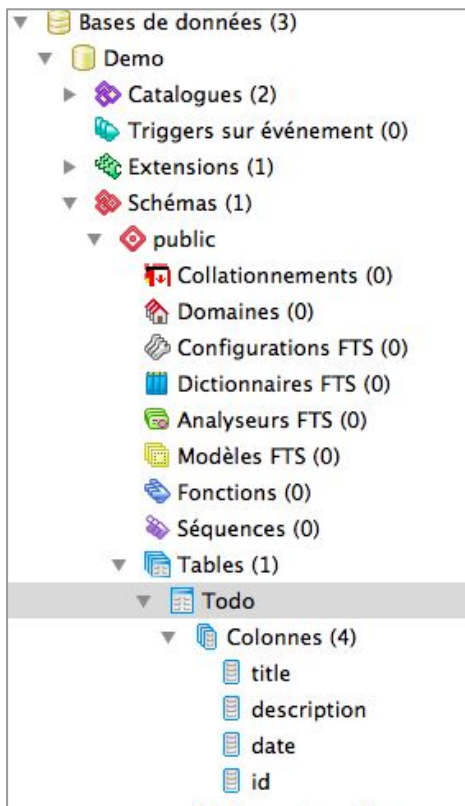
- chaque tache aura un titre, une description et une date

Modèle de données

Pour cette application, nous utiliserons la base de données postgresql. Nous n'avons besoin que d'une seule table : la table "todo" avec comme colonnes :

Nom de la colonne	Type
id (la clé primaire)	Int
title	varchar(64)
description	varchar(256)
date	date

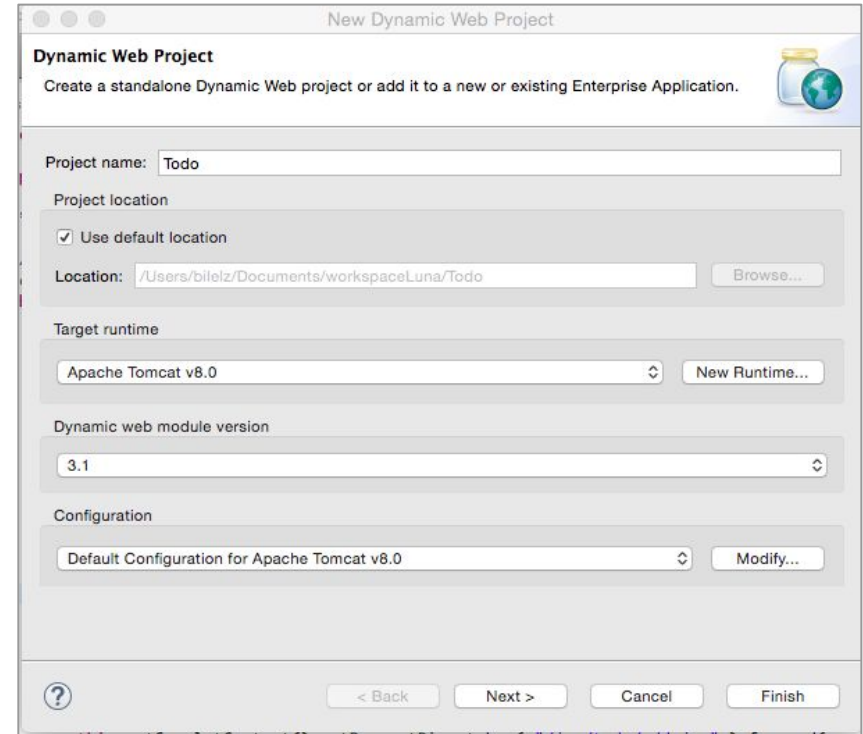
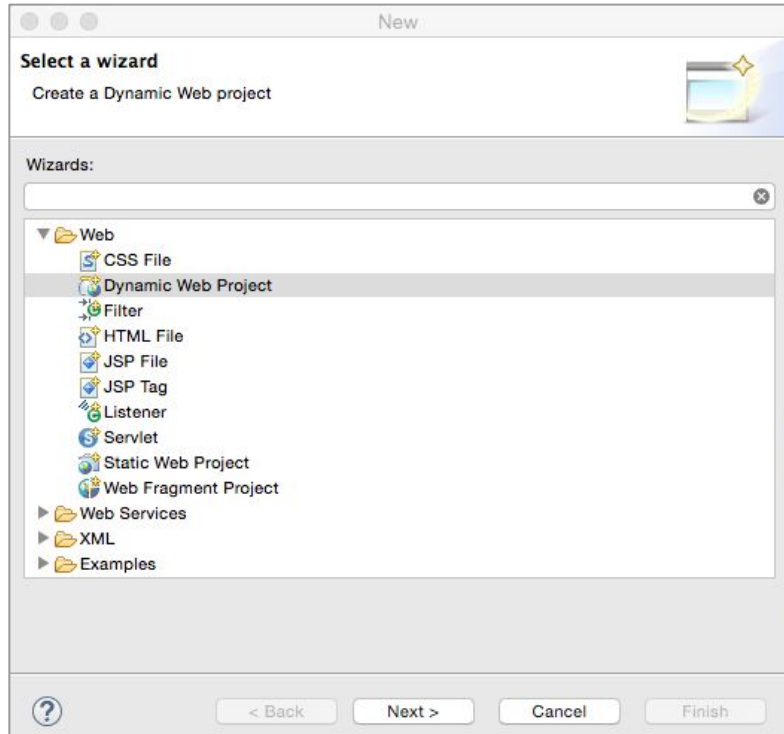
Modèle de données



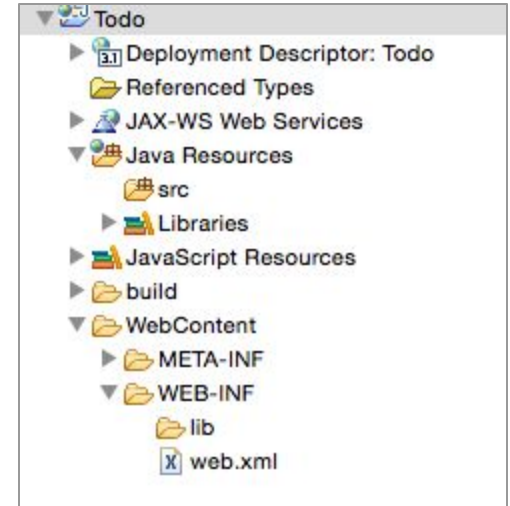
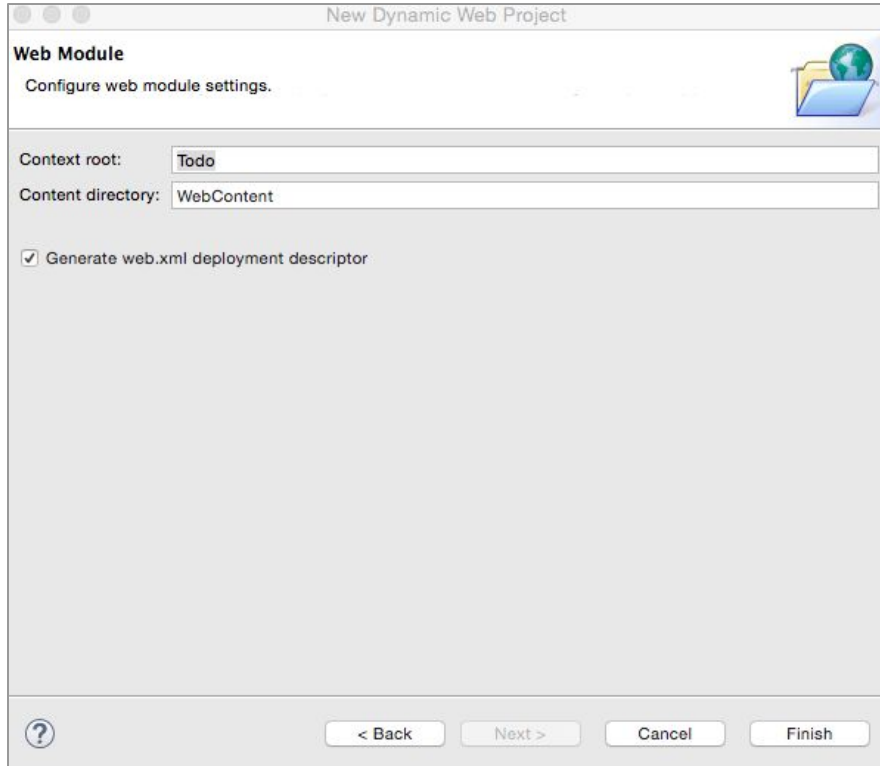
```
CREATE TABLE "Todo"  
(  
    title character varying(64) NOT NULL,  
    description character varying(256) NOT NULL,  
    date date, -- date de derniere mise à jour  
    id serial NOT NULL,  
    CONSTRAINT primarykey PRIMARY KEY (id)  
)
```


Création du projet avec Eclipse

Dans Eclipse, aller dans File/New project/Others



Création du projet avec Eclipse

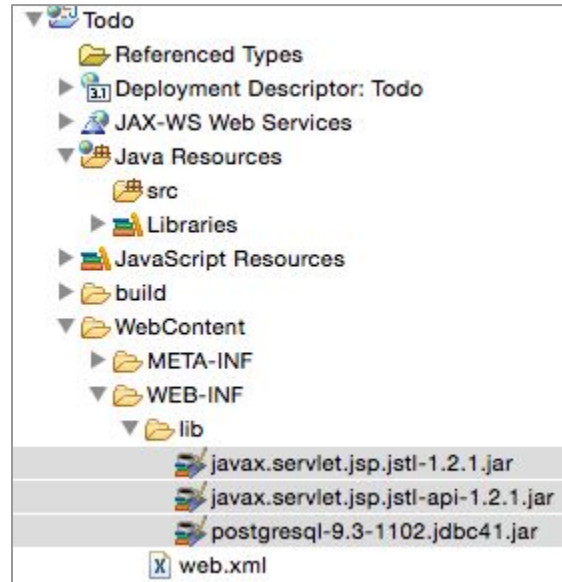


Arborescence du projet

On coche "Generate web.xml..."

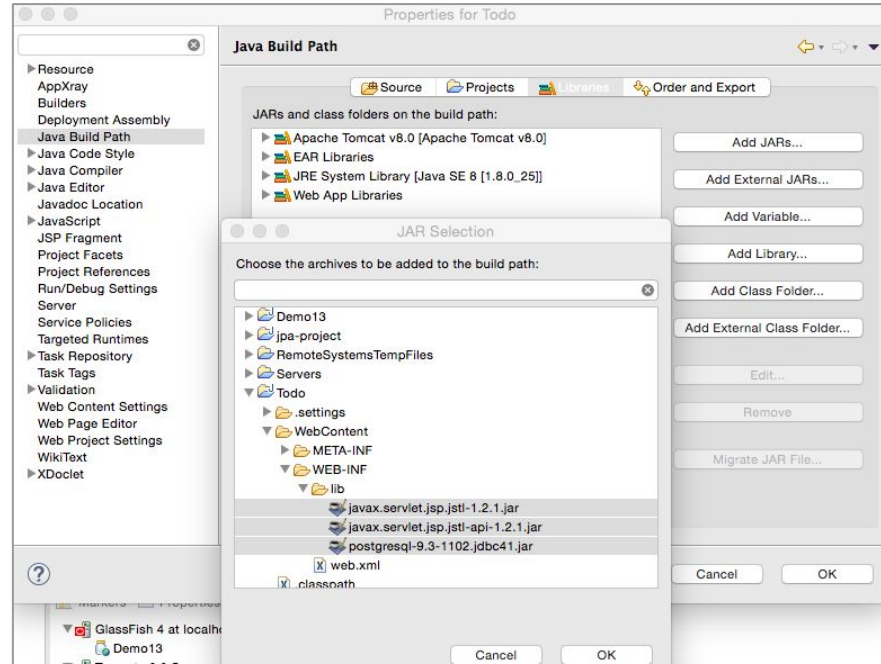
Installation des librairies

Téléchargement des librairies [JDBC](#) et JSTL([Core](#) + [API](#)) dans le répertoire “lib”

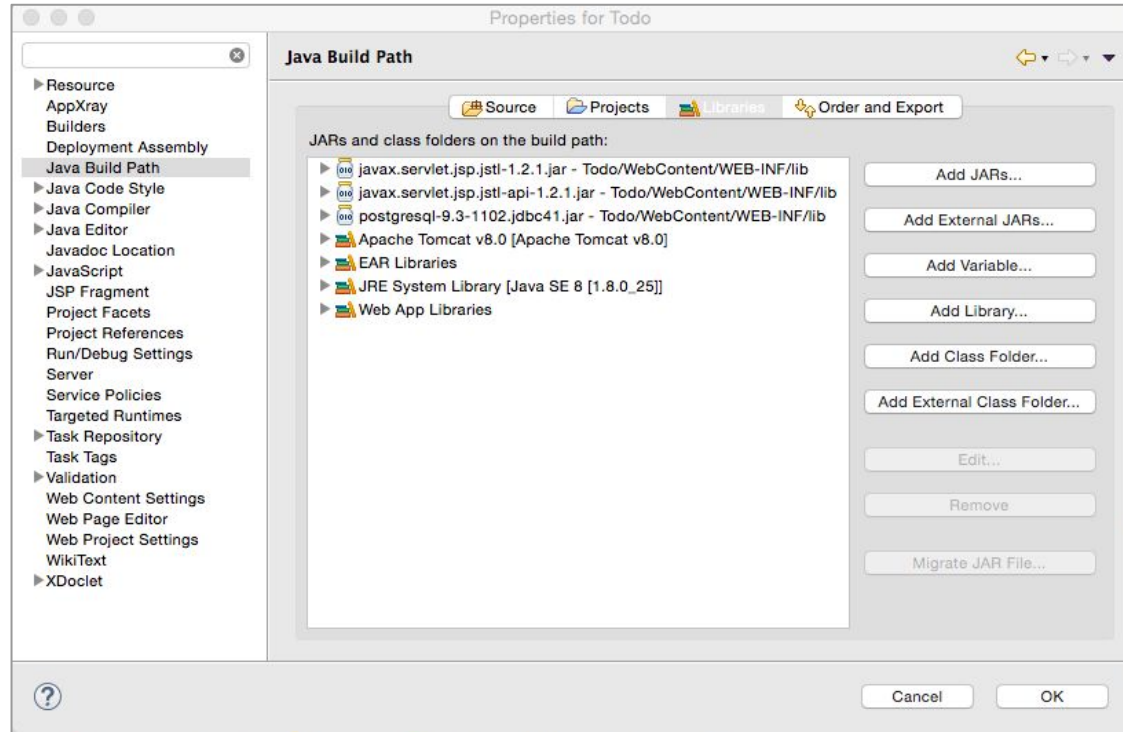


Installation des librairies

Pour que la projet puisse utiliser ces librairies, il faut les référencer dans le Java Build Path via un click droit sur le projet > “Propriétés >” Java Build path” > “Add Jars”

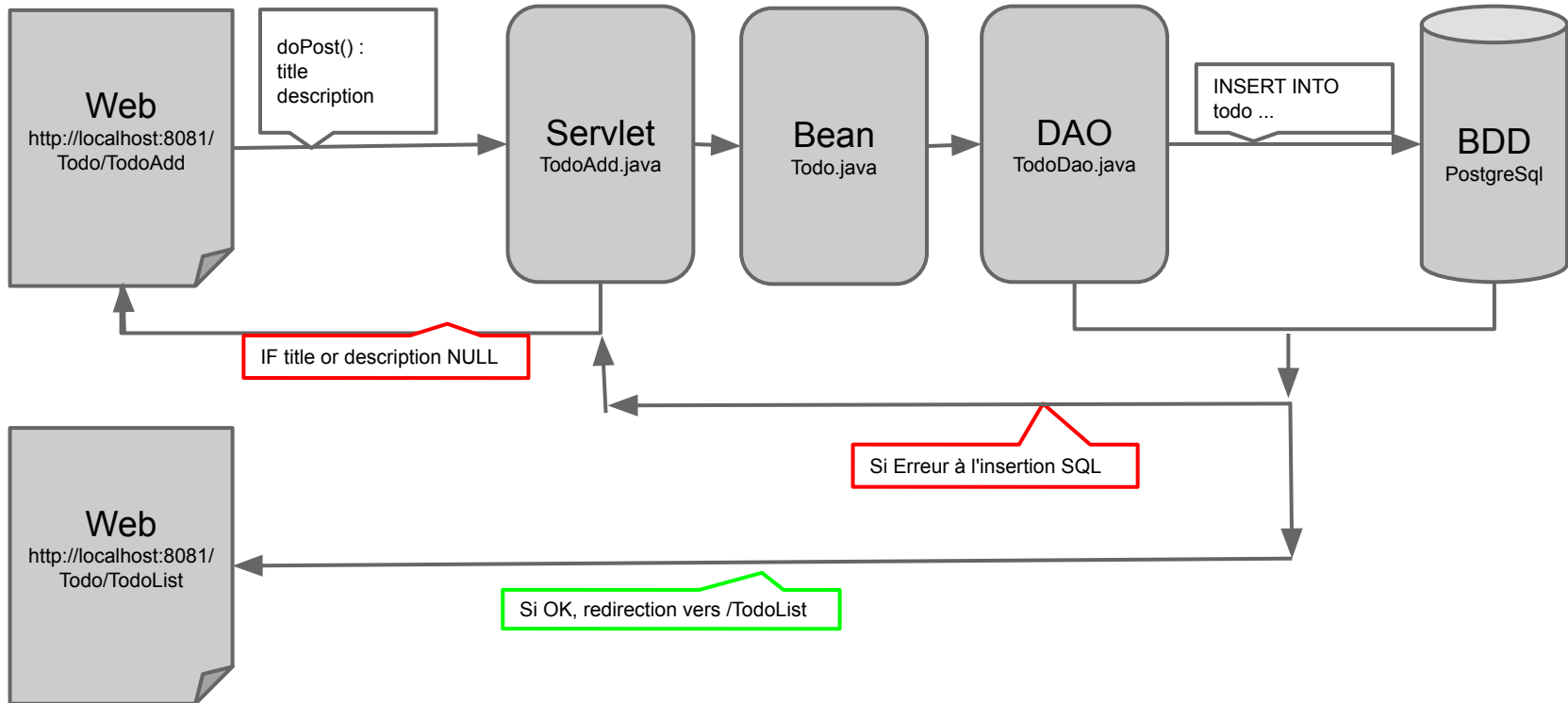


Installation des librairies



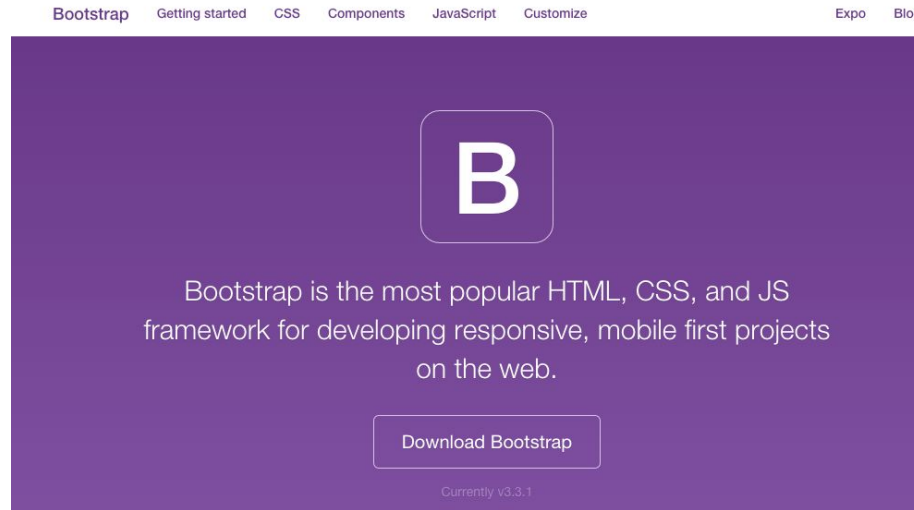
Workflow de la création d'un item

Soumission du formulaire



Créer de la page web TodoAdd.jsp

Pour écrire nos pages web, nous utiliserons le framework HTML [Bootstrap](#).



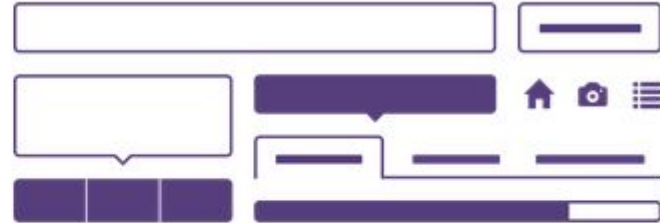
“Originally created by a designer and a developer at Twitter, Bootstrap has become one of the most popular front-end frameworks and open source projects in the world.”

Créer de la page web TodoAdd.jsp



One framework, every device.

Bootstrap easily and efficiently scales your websites and applications with a single code base, from phones to tablets to desktops with CSS media queries.



Full of features

With Bootstrap, you get extensive and beautiful documentation for common HTML elements, dozens of custom HTML and CSS components, and awesome jQuery plugins.

Créer de la page web TodoAdd.jsp

Pour l'écriture, nous allons partir du template pour les formulaires de Bootstrap
<http://getbootstrap.com/css/#forms>

```
32 <h1>Add item</h1>
33
34 <c:if test="${requestScope.message != null}">
35   <div class="alert alert-warning" role="alert">
36     ${requestScope.message}
37   </div>
38 </c:if>
39
40 <form role="form" method="post" action="" name="todoForm" class="well">
41   <div class="form-group">
42     <label for="title">Title</label>
43     <input type="text" class="form-control" name="title" id="title" placeholder="Enter title" value="${todo.title}">
44   </div>
45   <div class="form-group">
46     <label for="description">Description</label>
47     <input type="text" class="form-control" name="description" id="description" placeholder="Description" value="${todo.description}">
48   </div>
49
50   <button type="submit" class="btn btn-primary">Save</button>
51 </form>
```

Si la servlet, envoie un "message"
on teste sa valeur et on l'affiche
par ex: "Le titre ne peut pas être vide"

valorisation du champ de
saisie

Créer de la page web TodoAdd.jsp

localhost:8081/Todo/TodoAdd

TodoApp +

Home / Create

Add item

Title

Description

Save

Fil d'ariane

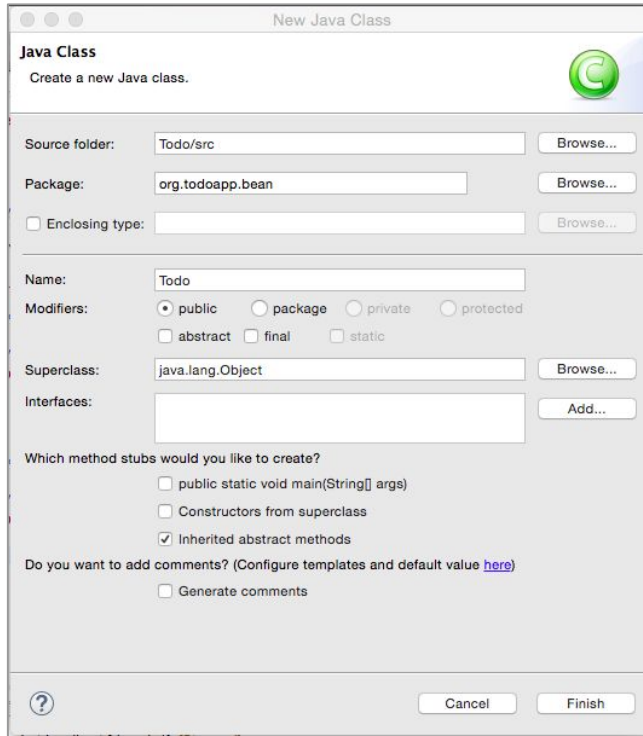
Champ de saisie "title"

Champ de saisie "description"

Bouton de validation du formulaire

Création du bean Java

L'application TodoList ne contiendra qu'un seul bean : todo



The screenshot shows the 'New Java Class' dialog box. The title bar says 'New Java Class'. Inside, there's a 'Java Class' section with a green 'C' icon and the text 'Create a new Java class.'. Below this, there are fields for 'Source folder:' (set to 'Todo/src'), 'Package:' (set to 'org.todoapp.bean'), and 'Enclosing type:' (empty). The 'Name:' field is set to 'Todo'. The 'Modifiers:' section has radio buttons for 'public' (selected), 'package', 'private', and 'protected', and checkboxes for 'abstract', 'final', and 'static'. The 'Superclass:' field is set to 'java.lang.Object'. The 'Interfaces:' field is empty. There's a section 'Which method stubs would you like to create?' with checkboxes for 'public static void main(String[] args)', 'Constructors from superclass', and 'Inherited abstract methods' (checked). At the bottom, there's a question 'Do you want to add comments? (Configure templates and default value [here](#))' with a checkbox for 'Generate comments'. The bottom of the dialog has 'Cancel' and 'Finish' buttons.



Créer du bean Java

```
1 package org.todoapp.bean;
2
3 import java.sql.Date;
4
5 public class Todo {
6
7     private Integer id;
8     private String title;
9     private String description;
10    private Date date;
11
12    public Integer getId() {
13        return id;
14    }
15    public void setId(Integer id) {
16        this.id = id;
17    }
18    public String getTitle() {
19        return title;
20    }
21    public void setTitle(String title) {
22        this.title = title;
23    }
24    public String getDescription() {
25        return description;
26    }
27    public void setDescription(String description) {
28        this.description = description;
29    }
30    public Date getDate() {
31        return date;
32    }
33    public void setDate(Date date) {
34        this.date = date;
35    }
36
37 }
```

Création de DAO

New Java Class

Java Class
Create a new Java class.

Source folder:

Package:

☐ Enclosing type:

Name:

Modifiers: ☒ public ☐ package ☐ private ☐ protected
☒ abstract ☐ final ☐ static

Superclass:

Interfaces:

Which method stubs would you like to create?

☐ public static void main(String[] args)
☐ Constructors from superclass
☒ Inherited abstract methods

Do you want to add comments? (Configure templates and default value [here](#))
☐ Generate comments

```
6 public abstract class DAO<T> {
7
8     protected Connection connect = null;
9
10    /**
11     * Constructeur
12     * @param conn
13     */
14    public DAO(Connection conn){
15        this.connect = conn;
16    }
17
18    /**
19     * Methode de creation d'item
20     * @param obj
21     * @return
22     */
23    public abstract boolean create(T obj);
24    /**
25     * Methode pour effacer un item
26     * @param obj
27     * @return
28     */
29    public abstract boolean delete(T obj);
30    /**
31     * Methode de mise à jour
32     * @param obj
33     * @return
34     */
35    public abstract boolean update(T obj);
36    /**
37     * Methode de recherche d'un item
38     * @param id
39     * @return
40     */
41    public abstract T find(int id);
42
43    /**
44     * Methode de recherche de tous les items
45     * @return ArrayList
46     */
47    public abstract ArrayList<T> list();
48 }
```

TodoDAO - CRUD

Méthode Create

```
11 public class TodoDao extends DAO<Todo>{
12
13     public TodoDao(Connection conn) {
14         super(conn);
15     }
16
17     public boolean create(Todo obj) {
18
19         try {
20             //Vu que nous sommes sous postgres, nous allons chercher manuellement
21             //la prochaine valeur de la séquence correspondant à l'id de notre table
22             ResultSet result = this .connect
23                 .createStatement(ResultSet.TYPE_SCROLL_INSENSITIVE, ResultSet.CONCUR_UPDATABLE)
24                 .executeQuery("select nextval('\"\"Todo_id_seq\"\"') as id");
25
26             if(result.first()){
27                 int id = result.getInt("id");
28                 PreparedStatement prepare = this.connect
29                     .prepareStatement("INSERT INTO todo (id, title, description, date) VALUES(?, ?, ?, ?)");
30
31                 prepare.setLong(1, id);
32                 prepare.setString(2, obj.getTitle());
33                 prepare.setString(3, obj.getDescription());
34                 prepare.setDate(4, obj.getDate());
35
36                 prepare.executeUpdate();
37
38                 obj = this.find(id);
39             }
40         } catch (SQLException e) {
41             e.printStackTrace();
42         }
43         return true;
44     }
45 }
```

TodoDAO - CRUD

Méthode Find

```
55 public Todo find(int id) {  
56     Todo todo = new Todo();  
57  
58     try {  
59         ResultSet result = this.connect.createStatement(ResultSet.TYPE_SCROLL_INSENSITIVE, ResultSet.CONCUR_READ_ONLY)  
60             .executeQuery("SELECT * FROM todo WHERE id = " + id);  
61  
62         if (result.first())  
63             todo = new Todo( result.getInt("id"), result.getString("name"), result.getString("description"), result.getDate("date"));  
64     } catch (SQLException e) {  
65         e.printStackTrace();  
66     }  
67     return todo;  
68 }  
69 }
```

Méthode List

```
71 public ArrayList<Todo> list() {  
72     ArrayList<Todo> todoList = new ArrayList<Todo>();  
73     Todo todo = new Todo();  
74  
75     try {  
76         ResultSet result = this.connect.createStatement(ResultSet.TYPE_SCROLL_INSENSITIVE, ResultSet.CONCUR_READ_ONLY)  
77             .executeQuery("SELECT * FROM Todo");  
78  
79         while(result.next()){  
80             todo = new Todo( result.getInt("id"), result.getString("name"), result.getString("description"), result.getDate("date"));  
81             todoList.add(todo);  
82         }  
83     } catch (SQLException e) {  
84         e.printStackTrace();  
85     }  
86     return todoList;  
87 }  
88  
89 }
```

TodoDAO - CRUD

Méthode Update

```
86 public boolean update(Todo obj) {  
87     try {  
88  
89         PreparedStatement prepare = this.connect  
90             .prepareStatement("UPDATE todo SET title = ?, description = ?, date = ? where id = ?");  
91  
92         prepare.setString(1, obj.getTitle());  
93         prepare.setString(2, obj.getDescription());  
94         prepare.setDate(3, obj.getDate());  
95         prepare.setLong(4, obj.getId());  
96  
97         prepare.executeUpdate();  
98  
99     } catch (SQLException e) {  
100         e.printStackTrace();  
101     }  
102  
103     return true;  
104 }
```

TodoDAO.java

TodoDAO - CRUD

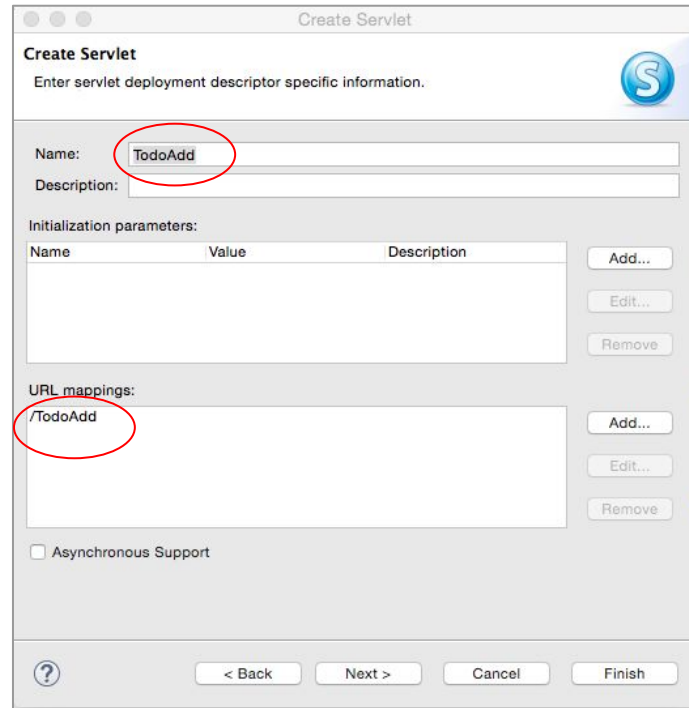
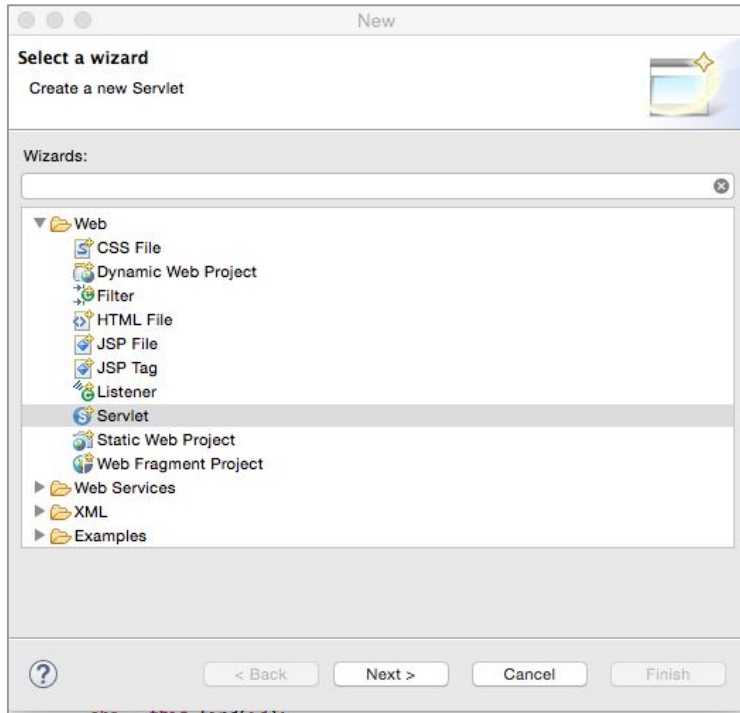
Méthode Delete

```
45 public boolean delete(Todo obj) {  
46     try {  
47         this.connect  
48         .createStatement(ResultSet.TYPE_SCROLL_INSENSITIVE, ResultSet.CONCUR_UPDATABLE)  
49         .executeUpdate("DELETE FROM todo WHERE id = " + obj.getId());  
50  
51     } catch (SQLException e) {  
52         e.printStackTrace();  
53     }  
54     return true;  
55 }
```

TodoDAO.java

Création d'une servlet : TodoAdd

Création de la première servlet qui va créer une tâche

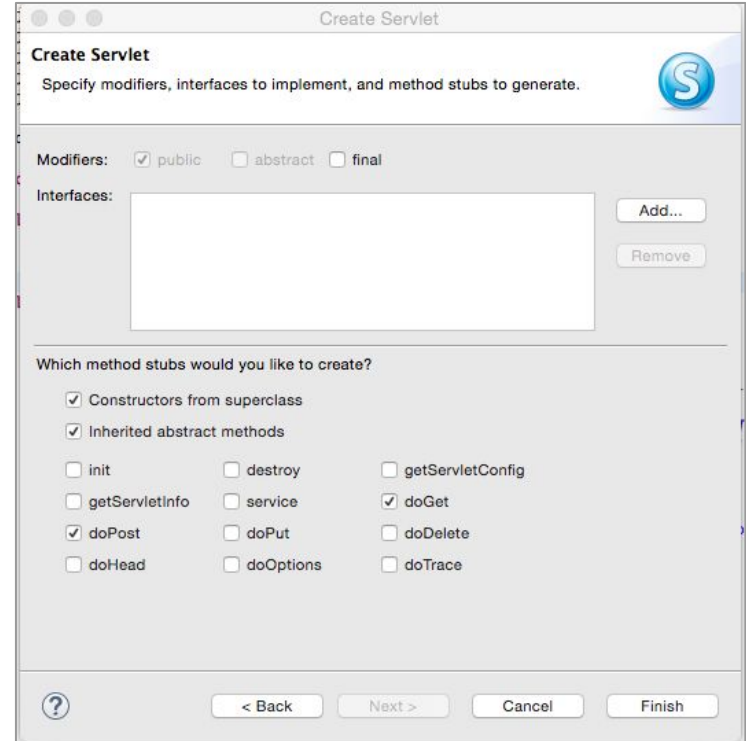


Création d'une servlet : TodoAdd

Par défaut Eclipse propose d'implémenter les méthodes:

doGet : pour gérer les appels à la servlet directement depuis l'URL /TodoAdd

doPost : pour gérer les appels à la servlet depuis une soumission de formulaire vers l'URL /TodoAdd



Contenu de la servlet : TodoAdd

doGet() : Cette appelé quand l'utilisateur se rend à l'URL
http://localhost:8081/Todo/TodoAdd

```
21 @WebServlet("/TodoAdd")
22 public class TodoAdd extends HttpServlet {
23     private static final long serialVersionUID = 1L;
24
25     /**
26      * @see HttpServlet#HttpServlet()
27      */
28     public TodoAdd() {
29         super();
30     }
31
32     /**
33      * @see HttpServlet#doGet(HttpServletRequest request, HttpServletResponse response)
34      */
35     protected void doGet(HttpServletRequest request, HttpServletResponse response) throws ServletException, IOException {
36         // TODO Auto-generated method stub
37         // passage de l'affichage vers la JSP
38         this.getServletContext().getRequestDispatcher( "/jsp/TodoAdd.jsp" ).forward( request, response );
39     }
```

Contenu de la servlet : TodoAdd

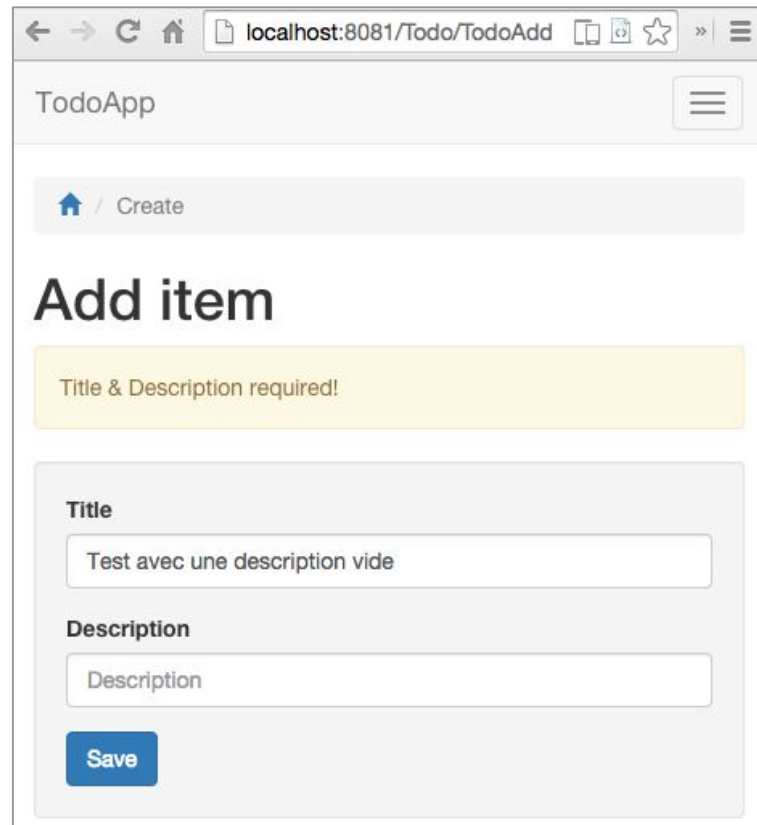
doPost() :
Cette méthode
est appelée quand
l'utilisateur valide
le formulaire

```
41- /**
42-  * @see HttpServlet#doPost(HttpServletRequest request, HttpServletResponse response)
43-  */
44- protected void doPost(HttpServletRequest request, HttpServletResponse response) throws ServletException, IOException {
45-
46-     Todo todo = new Todo();
47-     todo.setTitle(request.getParameter("title"));
48-     todo.setDescription(request.getParameter("description"));
49-
50-     // validation du formulaire, title & description sont obligatoires
51-     if( request.getParameter("title") == null || request.getParameter("title").length() == 0
52-         || request.getParameter("description") == null || request.getParameter("description").length() == 0){
53-
54-         request.setAttribute("message", "Title & Description required!");
55-         request.setAttribute("todo", todo);
56-         this.getServletContext().getRequestDispatcher( "/jsp/TodoAdd.jsp" ).forward( request, response );
57-
58-     }else{
59-
60-         try{
61-             DAO<Todo> todoDAO = new TodoDao(ConnectionDB.getInstance());
62-             todo = todoDAO.create(todo);
63-             if(todo.getId() != null){
64-                 request.setAttribute("message", "Item created!");
65-                 request.setAttribute("todoIdCreated", todo.getId());
66-             }
67-
68-         } catch (Exception e) {
69-             request.setAttribute("message", "Error exception");
70-             this.getServletContext().getRequestDispatcher( "/jsp/TodoAdd.jsp" ).forward( request, response );
71-             e.printStackTrace();
72-         }
73-
74-         this.getServletContext().getRequestDispatcher( "/TodoList" ).forward( request, response );
75-     }
76- }
77 }
```

TodoAdd : validation de formulaire

```
32 <h1>Add item</h1>
33
34 <c:if test="${requestScope.message != null}">
35   <div class="alert alert-warning" role="alert">
36     ${requestScope.message}
37   </div>
38 </c:if>
39
40 <form role="form" method="post" action="" name="todoForm" class="well">
41   <div class="form-group">
42     <label for="title">Title</label>
43     <input type="text" class="form-control" name="title"
44           id="title" placeholder="Enter title" value="${todo.title}">
45   </div>
46   <div class="form-group">
47     <label for="description">Description</label>
48     <input type="text" class="form-control" name="description"
49           id="description" placeholder="Description" value="${todo.description}">
50   </div>
51
52   <button type="submit" class="btn btn-primary">Save</button>
53 </form>
```

JSP



The screenshot shows a web browser at localhost:8081/ToDo/ToDoAdd. The page title is 'TodoApp'. Below the title bar, there is a navigation bar with a home icon and the text '/ Create'. The main heading is 'Add item'. A yellow alert box displays the message 'Title & Description required!'. Below this, there are two form fields: 'Title' with the placeholder text 'Test avec une description vide' and 'Description' with the placeholder text 'Description'. At the bottom of the form is a blue 'Save' button.

WEB

TodoAdd : validation avec HTML5

```
40 <form role="form" method="post" action="" name="todoForm" class="well">
41   <div class="form-group">
42     <label for="title">Title</label>
43     <input type="text" class="form-control" name="title" required
44           id="title" placeholder="Enter title" value="${todo.title}">
45   </div>
46   <div class="form-group">
47     <label for="description">Description</label>
48     <input type="text" class="form-control" name="description" required
49           id="description" placeholder="Description" value="${todo.description}">
50   </div>
51
52   <button type="submit" class="btn btn-primary">Save</button>
53 </form>
```

Attribut HTML5

Attribut HTML5

JSP

La validation en HTML5 permet d'éviter à l'utilisateur d'attendre une validation coté serveur.
Pour des raisons de sécurité, il faut tout de même garder une validation par la servlet.

TodoApp

Home / Create

Add item

Title

Description

Save

Création de la servlet TodoList.java

```
22 @WebServlet("/TodoList")
23 public class TodoList extends HttpServlet {
24     private static final long serialVersionUID = 1L;
25
26     /**
27      * @see HttpServlet#doGet(HttpServletRequest request, HttpServletResponse response)
28      */
29     protected void doGet(HttpServletRequest request, HttpServletResponse response) throws ServletException, IOException {
30         try{
31             // recherche en base de la liste de tous les éléments
32             DAO<Todo> todoDAO = new TodoDao(ConnectionDB.getInstance());
33             ArrayList<Todo> todoList = todoDAO.list();
34
35             // envoi de la liste "todoList" à JSP
36             request.setAttribute("todoList", todoList);
37
38             } catch (Exception e) {
39                 e.printStackTrace();
40             }
41
42             this.getServletContext().getRequestDispatcher( "/jsp/TodoList.jsp" ).forward( request, response );
43         }
44     }
```


Création de la servlet TodoList.java

```
71 public ArrayList<Todo> list() {  
72  
73     ArrayList<Todo> TodoList = new ArrayList<Todo>();  
74     Todo Todo = new Todo();  
75  
76     try {  
77         ResultSet result = this.connect.createStatement(ResultSet.TYPE_SCROLL_INSENSITIVE,ResultSet.CONCUR_READ_ONLY)  
78             .executeQuery("SELECT * FROM Todo");  
79  
80         while(result.next()){  
81             Todo = new Todo( result.getInt("id"), result.getString("name"), result.getString("description"), result.getDate("date"));  
82             TodoList.add(Todo);  
83         }  
84     } catch (SQLException e) {  
85         e.printStackTrace();  
86     }  
87  
88     return TodoList;  
89 }
```

DAO

TodoList.jsp

Pour cette page, deux extensions de JSTL seront utilisées

```
<%@ taglib prefix="fn" uri="http://java.sun.com/jsp/jstl/functions" %>
```

Permet d'utiliser des fonctions de calcul sur les chaînes de caractères

```
<%@ taglib prefix="fmt" uri="http://java.sun.com/jsp/jstl/fmt" %>
```

Permet de formater des données numériques et des dates

Le détail des fonctions en français <http://adiguba.developpez.com/tutoriels/j2ee/jsp/jstl/>

TodoList.jsp

```
36 <h1>Todo list (${fn:length(todoList)})</h1>
37
38 <c:if test="${requestScope.message != null}">
39   <div class="alert alert-warning" role="alert">
40     ${requestScope.message}
41   </div>
42 </c:if>
43
44 <c:if test="${fn:length(todoList) eq 0}">
45   <div class="alert alert-warning" role="alert">
46     No item.
47     <a href="/TodoAdd">
48       Create one.
49     </a>
50   </div>
51 </c:if>
52
53 <div class="list-group">
54   <c:forEach var="item" items="${todolist}">
55     <a href="/TodoRead?id=${item.id}" class="list-group-item" title="${item.description}">
56       ${item.title}
57
58       <div class="pull-right">
59         <fmt:formatDate type="date" value="${item.date}" />
60         <span class="glyphicon glyphicon-chevron-right" aria-hidden="true"></span>
61       </div>
62     </a>
63   </c:forEach>
64 </div>
```

Nb total d'éléments

Nb total d'éléments

Si il n'y aucun élément. On affiche un ce bloc

Boucle sur la liste avec mise en forme

Formatage de la date

TodoList.jsp

